A PRACTICAL ENGINEER'S GUIDE

Cloud Operations and Machine Learning

From Zero to Advanced — Concepts, Hands-On Examples, and Production Practice

Part I — Cloud Operations • Part II — Machine Learning • Part III — MLOps Capstone

October 2026

Contents

Part I — Cloud Operations: From Zero to Advanced

1 Cloud Computing Fundamentals 3

2 Core Building Blocks: Compute, Storage, Networking 4

3 Infrastructure as Code 6

4 Containers and Kubernetes 8

5 CI/CD Pipelines 9

6 Observability: Metrics, Logs, Traces 11

7 Security and Identity 12

8 Advanced Cloud Operations: SRE, GitOps, FinOps 13

Part II — Machine Learning: From Zero to Advanced

9 What Machine Learning Is 15

10 The Mathematics You Actually Need 16

11 Data Preparation and Your First Model 17

12 Evaluating Models Properly 19

13 Classic Algorithms and Ensembles 20

14 Neural Networks and Deep Learning 22

15 Advanced Architectures: CNNs, RNNs, Transformers 23

Part III — Where the Two Meet: MLOps

16 MLOps: Operating Models in Production 25

17 Capstone: An End-to-End Project 27

References 28

Part I — Cloud Operations: From Zero to Advanced

How modern infrastructure is provisioned, automated, secured, and kept running — starting from "what is the cloud?" and ending with the practices used by site reliability teams at scale.

1 Cloud Computing Fundamentals

Cloud computing is the on-demand delivery of computing resources — servers, storage, databases, networking, software — over the internet, billed by usage rather than owned outright. Instead of buying physical machines weeks in advance, you request a virtual server in seconds and pay only for what you consume. The U.S. National Institute of Standards and Technology (NIST) defines the model with five essential characteristics: on-demand self-service, broad network access, resource pooling, rapid elasticity, and measured service.[1]

1.1 Service models: IaaS, PaaS, SaaS

The cloud is offered in three classic layers. The key question each layer answers is: *how much of the stack do you manage yourself?*

Table 1. Service models and who manages what

Layer	What you get	You manage	Provider manages	Typical examples
IaaS (Infrastructure as a Service)	Virtual machines, disks, networks	OS, middleware, runtime, application, data	Hardware, virtualization, physical network	AWS EC2, Azure VMs, Google Compute Engine
PaaS (Platform as a Service)	Managed runtime to deploy code	Application code and data	Everything below the app	Heroku, Google App Engine, AWS Elastic Beanstalk
SaaS (Software as a Service)	Finished application in the browser	Configuration and your data	The entire stack	Gmail, Salesforce, Slack
Serverless / FaaS	Run functions on events, per-invocation billing	Function code only	Scaling, servers, runtime	AWS Lambda, Cloud Functions, Azure Functions

> **Definition (Elasticity).** *The ability of a system to acquire and release resources automatically so that capacity tracks demand closely — scaling out during a traffic spike and back in when it passes.*

1.2 Deployment models and global topology

Clouds are deployed as **public** (shared provider infrastructure, e.g. AWS, Azure, Google Cloud), **private** (dedicated to one organization, often built with OpenStack or VMware), or **hybrid** (workloads split between the two, connected by VPN or dedicated links).

Physically, each provider divides the world into **Regions** (geographic areas such as `us-east-1` or `europe-west1`) and **Availability Zones (AZs)** — one or more isolated data centers within a region with independent power and networking. The fundamental reliability pattern of the cloud is: *deploy across at least two AZs so that the failure of one data center does not take your service down*.

> **Example 1 — Thinking in failure domains**
>
> An online shop runs two copies of its web application: one in AZ A, one in AZ B, behind a load balancer, with a database that replicates synchronously between the zones. When AZ A loses power during a storm, the load balancer health checks fail for the affected servers and traffic shifts to AZ B within seconds. Customers notice nothing. This single design decision — multi-AZ — is the difference between an outage and a non-event.

1.3 Why organizations move to the cloud

- **CapEx to OpEx:** replace up-front hardware purchases with pay-as-you-go operating expense.
- **Speed:** a new environment that took 6 weeks to order and rack now takes minutes of API calls.
- **Global reach:** deploy close to users on another continent without building data centers.
- **Elastic economics:** pay for 100 servers for the 4 hours of a product launch, then 10 servers again.
- **Managed services:** offload undifferentiated work (database patching, queue clustering) to the provider.

2 Core Building Blocks: Compute, Storage, Networking

2.1 Compute

Cloud compute comes in four main forms, ordered from most control to least operational burden:

Table 2. Compute options compared

Option	Unit of work	Boot time	Best for	Ops burden
Virtual machines	Whole OS instance	Minutes	Lift-and-shift, legacy apps, full control	High (patching, AMI baking)
Containers	Process + filesystem image	Seconds	Microservices, portable workloads	Medium (orchestration)

Functions (FaaS)	Single event handler	Milliseconds (warm)	Event processing, glue code, spiky jobs	Low
Managed batch / jobs	Scheduled job	Seconds–minutes	ETL, rendering, ML training	Low–medium

2.2 Storage

Storage is chosen by access pattern, not by capacity:

- **Object storage** (AWS S3, GCS, Azure Blob): flat namespace of immutable objects behind an HTTP API; effectively infinite, extremely durable (providers quote "eleven nines" by replicating across AZs). Ideal for files, backups, data lakes, model artifacts.
- **Block storage** (EBS, Persistent Disk): a virtual hard drive attached to one VM; needed for databases and anything that wants a filesystem with low latency.
- **File storage** (EFS, Filestore): shared POSIX filesystem mounted by many machines at once.
- **Databases as a service:** relational (RDS, Cloud SQL), key-value/document (DynamoDB, Firestore), and in-memory caches (ElastiCache/Memorystore running Redis).

2.3 Networking: the VPC

A **Virtual Private Cloud (VPC)** is your own isolated software-defined network inside the provider. You define IP ranges (`10.0.0.0/16`), carve them into **subnets**, and control traffic with route tables, security groups (instance-level stateful firewalls), and network ACLs. The standard layout is *public subnets* (reachable from the internet, hosting load balancers) and *private subnets* (no inbound internet route, hosting application servers and databases).

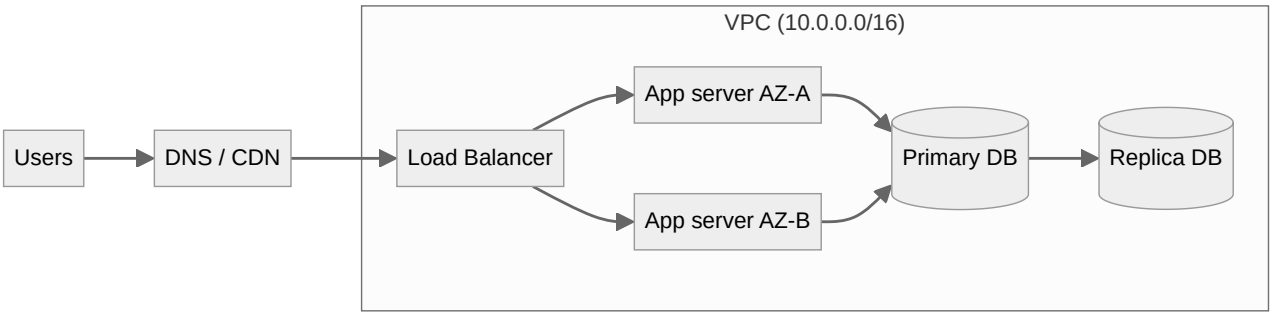


Figure 1. A minimal production-grade VPC layout: public entry through a load balancer, private application and database tiers across two availability zones.

2.4 Autoscaling in practice

Autoscaling groups watch a metric (CPU, queue depth, request count) and add or remove instances to hold it near a target. Figure 2 shows simulated behavior over a day: capacity follows demand, including a midday flash spike.

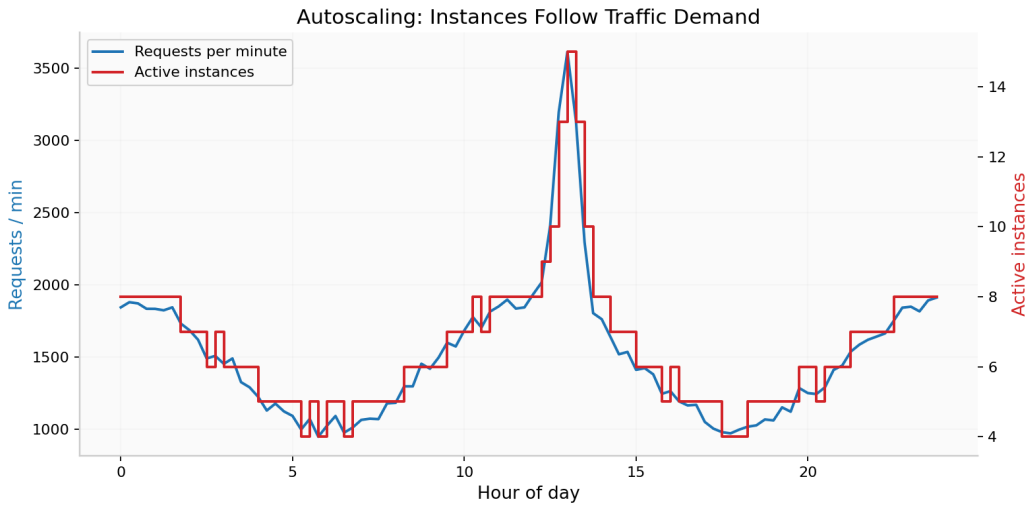


Figure 2. Autoscaling: instance count (right axis) tracks traffic (left axis), absorbing a flash spike around 13:00 without manual intervention.

3 Infrastructure as Code

Infrastructure as Code (IaC) means defining infrastructure — networks, servers, databases, permissions — in declarative text files stored in version control, rather than clicking through a web console. The benefits are reviewability (infrastructure changes go through pull requests), repeatability (rebuild an environment exactly), and auditability (`git log` is your change history).

> **Definition (Idempotence).** *Applying the same configuration twice produces the same result as applying it once. IaC tools achieve this by comparing desired state with actual state and making only the necessary changes.*

3.1 Terraform: a complete example

Terraform is the most widely used multi-cloud IaC tool; it expresses infrastructure in HCL (HashiCorp Configuration Language).[2] The following creates the two-tier layout from Figure 1 on AWS:

main.tf

```
provider "aws" { region = "eu-west-1" }

resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
  tags       = { Name = "shop-vpc" }
}

resource "aws_subnet" "public" {
  count             = 2                       # one per AZ
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.${count.index}.0/24"
  availability_zone = data.aws_availability_zones.all.names[count.index]
}

resource "aws_security_group" "app" {
  vpc_id = aws_vpc.main.id
  ingress {                     # only the LB may reach app servers
    from_port       = 8080
    to_port         = 8080
    protocol        = "tcp"
    security_groups = [aws_security_group.lb.id]
  }
}

resource "aws_autoscaling_group" "app" {
  min_size         = 2
  max_size         = 10
  desired_capacity = 2
  vpc_zone_identifier = aws_subnet.public[*].id
  launch_template {
    id      = aws_launch_template.app.id
    version = "$Latest"
  }
}
```

The workflow is always the same three commands: `terraform init` (download providers), `terraform plan` (preview the diff between code and reality), `terraform apply` (make reality match the code). State — the record of what Terraform manages — is stored remotely (e.g. S3 with locking) so the whole team shares one view.

3.2 Choosing an IaC tool

Table 3. IaC tool landscape

Tool	Style	Scope	Notes
Terraform / OpenTofu	Declarative (HCL)	Multi-cloud	Largest ecosystem of providers
AWS CloudFormation / CDK	Declarative / code (TypeScript, Python)	AWS only	Deep AWS integration
Pulumi	Real programming languages	Multi-cloud	Loops, functions, unit tests feel natural

Ansible	Procedural (YAML playbooks)	Config management	Best for configuring what runs *on* machines

4 Containers and Kubernetes

4.1 Why containers

A **container** packages an application with its libraries and runtime into a single image that runs identically on a laptop, a test server, and the cloud. Containers share the host kernel (unlike VMs, which each run a full OS), so they start in milliseconds and pack densely. Docker made the format ubiquitous; the Open Container Initiative (OCI) standardized it.

Dockerfile — packaging a small Python API

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8080
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8080"]
```

Build and run it: `docker build -t myapi:1.0 .` then `docker run -p 8080:8080 myapi:1.0`. Push to a registry (`docker push` to ECR/GCR/Docker Hub) and any machine can run the exact same artifact.

4.2 Kubernetes: orchestration

Running one container is easy; running hundreds across a fleet of machines — restarting failures, balancing load, rolling out new versions without downtime — is what **Kubernetes** (K8s) automates. Kubernetes is the de-facto standard container orchestrator, originally designed at Google and now governed by the CNCF.[3] Its core objects:

Table 4. Essential Kubernetes objects

Object	Purpose
Pod	Smallest deployable unit; one or more tightly coupled containers sharing network/storage
Deployment	Declares desired replica count and update strategy for Pods; performs rolling updates and rollbacks
Service	Stable virtual IP + DNS name that load-balances across a set of Pods
Ingress	HTTP routing from outside the cluster into Services (host/path rules, TLS)
ConfigMap / Secret	Configuration and credentials injected into Pods
HorizontalPodAutoscaler	Scales replica count on CPU/memory or custom metrics

deployment.yaml — desired state for our API

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: myapi }
spec:
  replicas: 3
  selector: { matchLabels: { app: myapi } }
  template:
    metadata: { labels: { app: myapi } }
    spec:
      containers:
        - name: myapi
          image: registry.example.com/myapi:1.0
          ports: [{ containerPort: 8080 }]
          resources:
            requests: { cpu: "250m", memory: "256Mi" }
            limits:   { cpu: "500m", memory: "512Mi" }
          readinessProbe:
            httpGet: { path: /health, port: 8080 }
---
apiVersion: v1
kind: Service
metadata: { name: myapi }
spec:
  selector: { app: myapi }
  ports: [{ port: 80, targetPort: 8080 }]
  type: ClusterIP
```

`kubectl apply -f deployment.yaml` tells the cluster the *desired* state; controllers then work continuously to make the actual state match — replacing crashed Pods, rescheduling around failed nodes, and rolling out image updates a few Pods at a time. This **reconciliation loop** is the central idea of Kubernetes and of modern cloud operations generally: declare intent, let the system converge.

5 CI/CD Pipelines

Continuous Integration (CI) is the practice of merging code to a shared mainline frequently, with every merge verified by an automated build and test run. **Continuous Delivery/Deployment (CD)** extends this so that every verified change is packaged and either ready to release (delivery) or released automatically (deployment). The practice was popularized by the DevOps movement and codified in *Continuous Delivery* by Humble and Farley.[4]

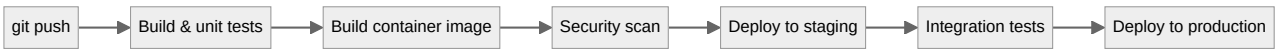


Figure 3. A standard CI/CD pipeline. Every stage is automated; a failure at any stage stops the pipeline and notifies the author.

5.1 A working pipeline in GitHub Actions

.github/workflows/ci-cd.yml

```
name: ci-cd
on:
  push: { branches: [main] }
  pull_request: {}

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install -r requirements.txt
      - run: pytest -q                # unit tests gate everything
      - run: ruff check .             # lint

  build-and-push:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/build-push-action@v6
        with:
          push: true
          tags: registry.example.com/myapi:${{ github.sha }}

  deploy:
    needs: build-and-push
    runs-on: ubuntu-latest
    steps:
      - run: |
          kubectl set image deployment/myapi \
            myapi=registry.example.com/myapi:${{ github.sha }}
          kubectl rollout status deployment/myapi
```

Three ideas make this production-grade rather than a demo: tests *gate* the pipeline (nothing ships that fails tests), images are tagged with the commit SHA (every artifact is traceable to code), and deployment is a rolling update verified by `rollout status` (so a bad release is caught and can be rolled back with `kubectl rollout undo`).

5.2 Progressive delivery

- **Rolling update:** replace instances a few at a time; simple and built into Kubernetes Deployments.
- **Blue/green:** run old and new environments side by side; flip the load balancer when the new one is verified; instant rollback by flipping back.
- **Canary:** send 1–5% of traffic to the new version, compare error rates and latency, then ramp up automatically (tools: Argo Rollouts, Flagger).
- **Feature flags:** decouple deploy from release — ship code dark, enable per-user or per-cohort at runtime.

6 Observability: Metrics, Logs, Traces

You cannot operate what you cannot see. Observability rests on three pillars:

Table 5. The three pillars of observability

Pillar	What it is	Questions it answers	Common tools
Metrics	Numeric time series (CPU, latency, error rate)	"Is the system healthy? Is it getting worse?"	Prometheus, CloudWatch, Grafana
Logs	Timestamped event records	"What exactly happened at 14:03:12?"	ELK/OpenSearch, Loki, Cloud Logging
Traces	End-to-end path of one request across services	"Which of the 8 microservices made this request slow?"	OpenTelemetry, Jaeger, Tempo

6.1 The RED method and a Prometheus example

For every service, track **Rate** (requests/second), **Errors** (failed requests/second), and **Duration** (latency distribution). With Prometheus, a request-rate query looks like:

```
sum(rate(http_requests_total{job="myapi", status=~"5.."}[5m]))
  /
sum(rate(http_requests_total{job="myapi"}[5m]))
```

This computes the fraction of 5xx responses over a 5-minute window — the single most important number for an HTTP service. An alert rule pages the on-call engineer when it stays above, say, 1% for 10 minutes.

6.2 Instrumentation example

main.py — exposing metrics from a FastAPI service

```
from fastapi import FastAPI
from prometheus_client import Counter, Histogram, make_asgi_app

app = FastAPI()
REQS = Counter("http_requests_total", "Requests", ["status"])
LAT  = Histogram("http_request_duration_seconds", "Latency")

@app.middleware("http")
async def observe(request, call_next):
    with LAT.time():
        resp = await call_next(request)
    REQS.labels(status=resp.status_code).inc()
    return resp

app.mount("/metrics", make_asgi_app())   # Prometheus scrapes this
```

Good observability is designed, not bolted on: every deploy event is annotated on dashboards, every alert links to a runbook, and every incident ends with a blameless postmortem that improves either the system or the alerts.

7 Security and Identity

7.1 The shared responsibility model

Table 6. Shared responsibility: who secures what

Layer	IaaS	PaaS	SaaS
Your data & access policies	You	You	You
Application code	You	You	Provider
Runtime & middleware	You	Provider	Provider
Operating system	You	Provider	Provider
Physical hardware & facilities	Provider	Provider	Provider

Data classification, identity, and access rules are *always* your responsibility, in every model.

7.2 IAM and least privilege

Identity and Access Management (IAM) answers two questions: who are you (authentication) and what may you do (authorization). The governing principle is **least privilege**: every human and every workload gets the minimum permissions needed, for the minimum time, and nothing else. Practical rules that prevent most cloud breaches:

- Never use long-lived access keys in code or CI; use workload identities / OIDC federation and short-lived tokens.
- Enable MFA for all human accounts; lock down the root/owner account and use it for almost nothing.
- Prefer roles over users: attach a role to a VM or Pod instead of embedding credentials.
- Keep data private by default: block public access on storage buckets; the majority of cloud data leaks are public buckets, not exotic exploits.
- Encrypt in transit (TLS everywhere) and at rest (provider-managed or customer-managed keys).

Example 2 — Anatomy of a typical cloud breach (and the fix)

A developer hardcodes a cloud access key in a script, pushes it to a public Git repository, and a scanner finds it within minutes. Automated crypto-mining instances launch in the account. *Fixes, in order of leverage:* short-lived credentials instead of keys (the leaked token expires in minutes), least-privilege roles (even a valid token cannot create instances), secret-scanning in CI (the push is blocked), and billing anomaly alerts (a 40× spend spike pages someone).

8 Advanced Cloud Operations: SRE, GitOps, FinOps

8.1 Site Reliability Engineering and SLOs

Site Reliability Engineering (SRE), created at Google, applies software engineering to operations.[5] Its core contract with users is the **Service Level Objective (SLO)**: a measurable target such as "99.9% of requests succeed with latency under 300 ms over 30 days." The gap between 100% and the SLO is the **error budget** — 43 minutes of allowed downtime per month at 99.9%. While budget remains, teams ship fast; when it is exhausted, feature work pauses in favor of reliability work. The error budget turns the eternal "speed vs. stability" argument into arithmetic.

> **Definition (Toil).***Manual, repetitive, automatable operational work that scales linearly with service size. SRE practice is to eliminate toil with software, keeping it below roughly 50% of an engineer's time.*

8.2 GitOps

GitOps takes IaC one step further: the Git repository *is* the desired state of the cluster, and an in-cluster agent (Argo CD or Flux) continuously reconciles the live system against the repo. Deploys become pull requests; rollbacks become `git revert`; the audit trail is the commit history. Configuration drift — someone hot-fixing production by hand — is detected and corrected automatically.



Figure 4. GitOps control loop: Git is the single source of truth; the agent converges the cluster and reports drift.

8.3 FinOps: cloud cost as an engineering discipline

The cloud converts infrastructure into a variable cost, which means engineers' architectural choices *are* financial decisions. Core FinOps practices:

- **Tagging and allocation:** every resource carries team/service tags so spend is attributable; untagged spend is treated as a bug.
- **Right-sizing:** most fleets run at 10–20% CPU; downsize instance families and set resource requests honestly in Kubernetes.
- **Commitment discounts:** reserved instances / savings plans cut steady-state cost 30–60% versus on-demand.
- **Spot instances:** spare capacity at 60–90% discount for fault-tolerant work (batch, CI runners, stateless replicas).

- **Storage lifecycle policies:** automatically move cold objects to archive tiers.
- **Anomaly detection:** alert on sudden spend spikes before the invoice does.

8.4 Multi-cloud and disaster recovery

Advanced teams plan explicitly for failure of large scopes: an AZ (routine), a whole region (rare but real), or a provider (very rare). The standard DR vocabulary: **RPO** (Recovery Point Objective — how much data you may lose, i.e. replication lag) and **RTO** (Recovery Time Objective — how fast you must be back). Strategies form a cost/speed ladder: *backup & restore* (cheapest, hours) → *pilot light* (minimal core always on) → *warm standby* (scaled-down copy ready to scale up) → *active-active multi-region* (most expensive, near-zero RTO). Choose per service based on what downtime actually costs the business — and rehearse failovers, because an untested backup is not a backup.

Part II — Machine Learning: From Zero to Advanced

From "what is a model?" through the mathematics, classic algorithms, and evaluation discipline, up to deep learning — always with runnable code.

9 What Machine Learning Is

Machine learning is the discipline of writing programs that improve at a task through experience, rather than through explicit hand-coded rules. The classic formulation is Tom Mitchell's: a program learns from experience E with respect to task T and performance measure P if its performance on T, as measured by P, improves with E.[6] Instead of telling the computer *how* to recognize spam, you show it 100,000 emails labeled spam/not-spam and it derives the rules itself.

9.1 The three paradigms

Table 7. Machine learning paradigms

Paradigm	Data	Goal	Typical tasks	Example algorithms
Supervised learning	Labeled (input, answer) pairs	Predict the label for new inputs	Spam detection, price prediction, diagnosis	Linear/logistic regression, trees, SVM, neural nets
Unsupervised learning	Unlabeled inputs	Discover structure	Customer segmentation, anomaly detection, compression	k-means, PCA, autoencoders
Reinforcement learning	Rewards from an environment	Learn a policy that maximizes long-run reward	Game playing, robotics, resource scheduling	Q-learning, policy gradients

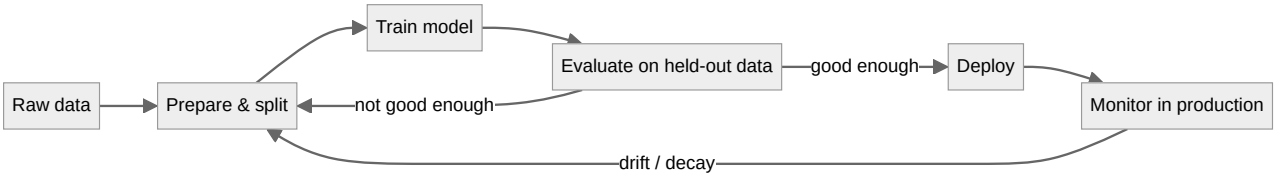


Figure 5. The machine learning lifecycle. Note the loop: ML is iterative, and deployment is the middle of the story, not the end.

9.2 Key vocabulary

- **Feature** (x): an input variable — e.g. the square footage of a house.
- **Label / target** (y): the thing to predict — e.g. the sale price.
- **Model**: a function $f_\theta(x)$ with tunable parameters θ, learned from data.
- **Training**: adjusting θ so predictions fit the data.
- **Generalization**: performing well on *unseen* data — the entire point.
- **Overfitting**: memorizing the training data instead of learning the pattern; great training score, poor real-world score.

10 The Mathematics You Actually Need

ML mathematics is large, but the working core is compact: vectors and matrices, a loss function, and gradient descent.

10.1 A model is a parameterized function

The simplest useful model, **linear regression**, predicts y as a weighted sum of features:

$$\hat{y} = f_\theta(x) = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b = \mathbf{w}^\top \mathbf{x} + b$$

Learning means choosing $\mathbf{w}$ and b. To choose them we must say what "good" means — a **loss function**. For[1] regression the standard choice is mean squared error (MSE) over n training examples:

$$J(\theta) = \frac{1}{n} \sum_{i=1}^{n} \left(\hat{y}^{(i)} - y^{(i)} \right)^2 \tag{2}$$

10.2 Gradient descent: how machines learn

$J(\theta)$ is a landscape; the best parameters sit at its lowest point. **Gradient descent** walks downhill: compute the gradient (the direction of steepest ascent) and step the other way, repeatedly:

$$\theta \leftarrow \theta - \alpha \, \nabla_\theta J(\theta)$$

The **learning rate** α controls step size: too small and training crawls; too large and it overshoots and diverges.[3] In practice we never compute the gradient over the whole dataset for every step — **stochastic/mini-batch gradient descent** estimates it from small random batches, which is both faster and a source of useful noise that helps escape poor local minima.

Example 3 — Gradient descent in 12 lines

```
import numpy as np

X = np.array([1.0, 2.0, 3.0, 4.0])      # feature: hours studied
y = np.array([2.0, 4.0, 6.0, 8.0])      # label: exam score (y = 2x)
w, b, lr = 0.0, 0.0, 0.05

for step in range(200):
    pred = w * X + b
    grad_w = (2/len(X)) * np.sum((pred - y) * X)   # dJ/dw
    grad_b = (2/len(X)) * np.sum(pred - y)         # dJ/db
    w -= lr * grad_w
    b -= lr * grad_b

print(f"w={w:.3f}, b={b:.3f}")   # w=2.000, b=0.000 (recovers the rule)
```

The model started knowing nothing (w = 0) and recovered the underlying rule $y = 2x$ purely by following the gradient. Every neural network on Earth — up to the largest language models — is trained by this same loop, just with billions of parameters and gradients computed by **backpropagation** instead of by hand.

10.3 Probability and information, briefly

Classification models output probabilities. The **sigmoid** function squashes any number into $(0, 1)$: $\sigma(z) = 1/(1 + e^{-z})$; its multi-class generalization is **softmax**. Training classifiers minimizes **cross-entropy loss**, $-\sum_i y_i \log \hat{p}_i$, which heavily punishes confident wrong answers. Concepts from information theory — entropy as uncertainty, KL divergence as distribution distance — recur throughout advanced ML.

11 Data Preparation and Your First Model

Real projects spend most of their effort on data, not algorithms — see Figure 6. Data work determines the ceiling of any model.

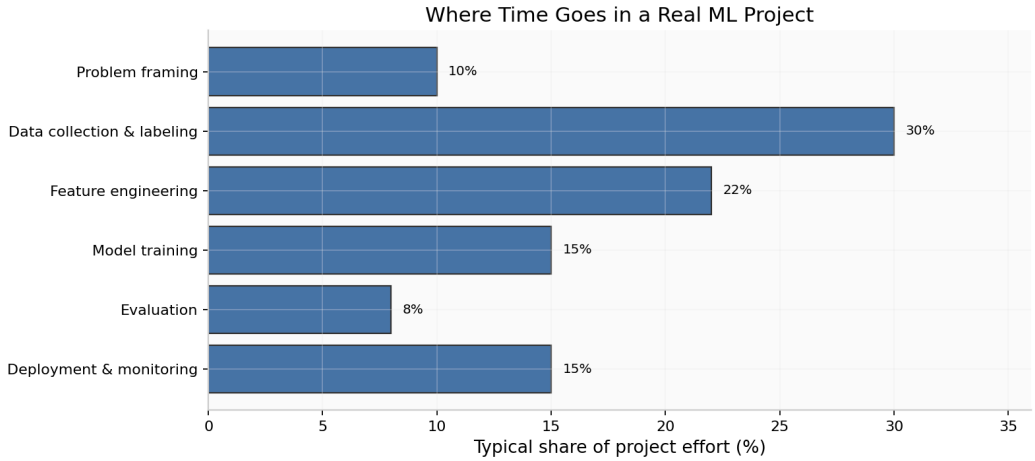


Figure 6. Typical effort distribution in applied ML projects: data work dwarfs modeling.

11.1 The data preparation checklist

1. **Audit the data:** distributions, missing values, duplicates, impossible values (age = 214), and label errors.
2. **Split before touching:** create train/validation/test sets *first*, and never let test data influence any decision — that is **data leakage**, the most common silent killer of ML projects.
3. **Handle missing values:** drop, impute with median/mode, or add "was missing" indicator features.
4. **Encode categories:** one-hot for few categories; embeddings or target encoding for many.
5. **Scale numerics:** standardize to zero mean / unit variance for gradient-based and distance-based models (fit the scaler on the training set only!).
6. **Engineer features:** domain knowledge encoded as new inputs (e.g. "days since last order") often beats a fancier algorithm.

11.2 A complete first pipeline with scikit-learn

scikit-learn is the standard Python library for classic ML.[7] This end-to-end example classifies iris flowers — the "hello world" of ML — using a Pipeline so that preprocessing is fitted only on training folds:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X, y = load_iris(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("clf",   LogisticRegression(max_iter=1000)),
])

# 5-fold cross-validation on the training set = honest estimate
scores = cross_val_score(pipe, X_tr, y_tr, cv=5)
print(f"CV accuracy: {scores.mean():.3f} +/- {scores.std():.3f}")

pipe.fit(X_tr, y_tr)
print(f"Held-out test accuracy: {pipe.score(X_te, y_te):.3f}")
```

Three habits in these 20 lines separate professionals from tutorials: *stratified* splitting keeps class proportions equal in both sets; *cross-validation* gives a stable estimate with an uncertainty interval; and the *Pipeline* makes leakage from the scaler impossible.

12 Evaluating Models Properly

12.1 Why accuracy lies

If 1% of transactions are fraudulent, a model that always predicts "legitimate" scores 99% accuracy while being useless. For imbalanced problems we need the confusion matrix and derived metrics:

Table 8. Confusion matrix

	Predicted positive	**Predicted negative**
Actually positive	TP — true positive	FN — false negative (missed)
Actually negative	FP — false positive (false alarm)	TN — true negative

$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall} = \frac{TP}{TP + FN} \qquad F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Precision answers "of everything I flagged, how much was real?" — it is the metric of false-alarm cost. **Recall**(4) answers "of everything real, how much did I catch?" — the metric of missed-case cost. A cancer screener optimizes recall; a court evidence system optimizes precision. The **ROC-AUC** summarizes the trade-off across all decision thresholds; **PR-AUC** is more informative under heavy class imbalance.

12.2 Bias, variance, and regularization

The total error of a model decomposes into **bias** (error from a too-simple model that misses the pattern — underfitting), **variance** (error from a too-flexible model that fits noise — overfitting), and irreducible noise. Figure 7 shows the trade-off.

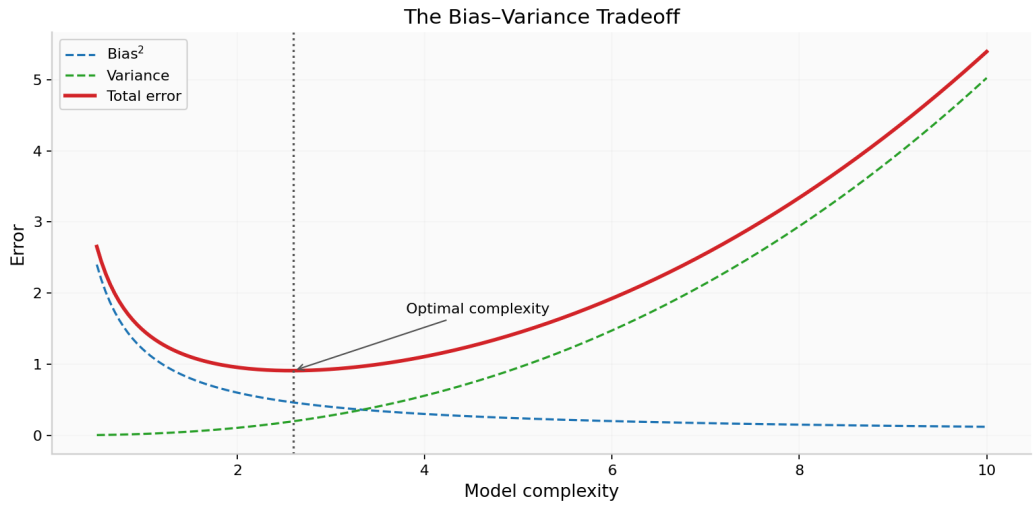


Figure 7. Bias–variance trade-off: total error is minimized at intermediate model complexity.

The practical toolkit for controlling complexity is **regularization** — deliberately constraining the model:

- **L2 (ridge)**: penalize large weights, $J' = J + \lambda \|\mathbf{w}\|^2$ — shrinks all weights smoothly.
- **L1 (lasso)**: penalize absolute weights, $J' = J + \lambda \|\mathbf{w}\|_1$ — drives useless feature weights to exactly zero (built-in feature selection).
- **Early stopping**: halt training when validation loss stops improving (see Figure 9 in Chapter 14).
- **More data / augmentation**: the strongest regularizer of all.

Example 4 — Diagnosing with learning curves

Plot training and validation score against training-set size. *Both scores low and close together* → high bias: add features or a more flexible model. *Large gap, training score near perfect* → high variance: get more data, regularize, or simplify. This one plot tells you which of the two fundamentally different problems you have — before you waste a week fixing the wrong one.

13 Classic Algorithms and Ensembles

13.1 A guided tour

Table 9. Classic algorithms at a glance

Algorithm	Idea	Strengths	Weaknesses
Linear / logistic regression	Weighted sum of features (+ sigmoid for classification)	Fast, interpretable, strong baseline	Only linear relationships
Decision tree	Series of if/else splits on features	Interpretable, handles mixed data, no scaling needed	High variance, overfits easily
Random forest	Many trees on random data/feature subsets; vote	Robust, few knobs, hard to badly misuse	Large, not interpretable, mediocre extrapolation
Gradient boosting (XGBoost, LightGBM)	Trees built sequentially, each fixing prior errors	State of the art on tabular data	Needs careful tuning; slower to train
Support vector machine	Widest-margin separating hyperplane (+ kernels)	Strong in high dimensions	Poor scaling to huge data; probabilities need calibration
k-means	Assign points to nearest of k centroids, iterate	Simple, fast clustering	Must choose k; assumes spherical clusters
PCA	Rotate axes to directions of max variance	Compression, visualization, denoising	Components lose interpretability

Figure 8 compares several classifiers on one tabular dataset — note that gradient boosting typically leads, but margins over a tuned random forest or even logistic regression can be small.

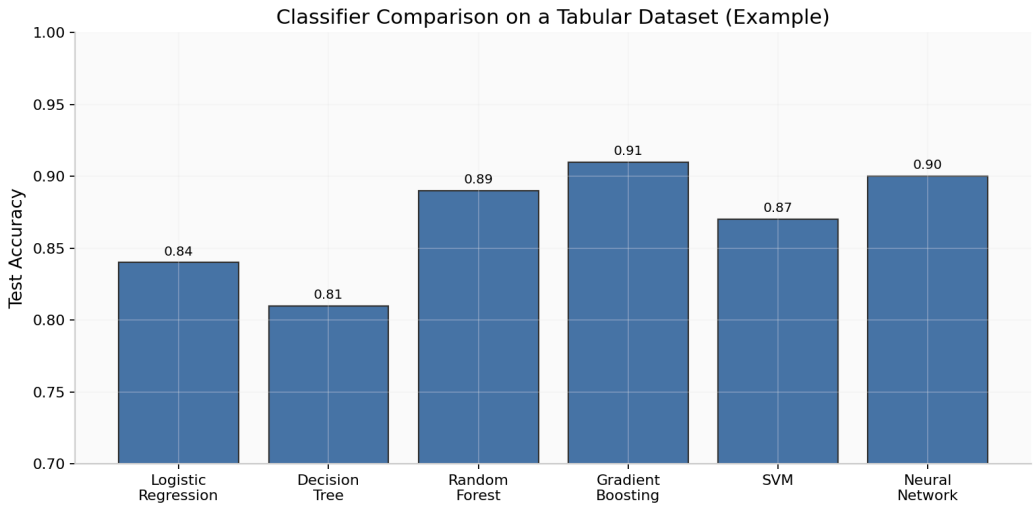


Figure 8. Classifier comparison on an example tabular dataset. Ensembles dominate, but simple models are often within a few points — and far easier to operate.

13.2 Why ensembles win: boosting in code

Bagging (random forest) reduces variance by averaging independent models; **boosting** reduces bias by training each new model on the residual errors of the ensemble so far. Gradient boosting on tabular data remains the default answer for most business ML problems:[8]

```
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.model_selection import GridSearchCV

gbm = HistGradientBoostingClassifier(random_state=42)
grid = GridSearchCV(gbm, {
    "learning_rate": [0.05, 0.1],
    "max_depth": [3, 5, None],
    "max_iter": [100, 300],
}, cv=5, scoring="f1", n_jobs=-1)

grid.fit(X_tr, y_tr)
print(grid.best_params_, grid.best_score_)
```

Hyperparameter search (grid/random/Bayesian) tunes the knobs that learning itself does not set — depth, learning rate, regularization strength. Always search on validation/cross-validation scores, never on the test set.

13.3 Rules of thumb for algorithm selection

1. Start with logistic regression or a small tree — a baseline you can explain.
2. Tabular data, medium size → gradient boosting. Almost always.
3. Images, audio, text → deep learning (Chapters 14–15).
4. Need interpretability for regulators → linear models, shallow trees, or SHAP explanations on top of ensembles.

5. Huge data, need online updates → linear models or streaming methods.

14 Neural Networks and Deep Learning

14.1 From linear models to networks

Stack several linear models with nonlinear *activation functions* between them and you get a **multilayer perceptron (MLP)** — a universal function approximator. Each layer computes:

$$\mathbf{h}^{(l)} = \phi\big(W^{(l)}\,\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\big), \qquad \phi \in \{\mathrm{ReLU}, \tanh, \dots\}$$

where $\mathrm{ReLU}(z) = \max(0, z)$ is the modern default. The magic is depth: early layers learn simple patterns(5) (edges, frequent words), deeper layers compose them into concepts (shapes, phrases, objects). Training is gradient descent (equation 3) with gradients propagated backward through all layers by the chain rule — **backpropagation**. The field's modern foundations are covered comprehensively by Goodfellow, Bengio and Courville.[9]

14.2 Training a network in PyTorch

```
import torch, torch.nn as nn

model = nn.Sequential(
    nn.Linear(4, 64), nn.ReLU(),
    nn.Linear(64, 32), nn.ReLU(),
    nn.Linear(32, 3),                    # 3 classes
)
opt  = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(100):
    for xb, yb in train_loader:          # mini-batches
        opt.zero_grad()
        loss = loss_fn(model(xb), yb)    # forward
        loss.backward()                  # backprop
        opt.step()                       # gradient step
```

Everything from Chapter 10 is visible here: a parameterized function, a loss, and gradient steps — the framework just computes $\nabla_\theta J$ automatically (`loss.backward()`).

14.3 Practical deep learning: what actually matters

- **Optimizers:** Adam/AdamW adapt the learning rate per parameter and are the safe default.
- **Learning rate schedule:** warm up, then decay; the LR is the single most impactful hyperparameter.
- **Batch normalization / layer normalization:** stabilize activations so deep stacks train at all.
- **Dropout:** randomly zero activations during training — a strong regularizer.

- **Weight decay:** L2 regularization applied in the optimizer.
- **Early stopping:** monitor validation loss and stop when it turns up — visible in Figure 9.

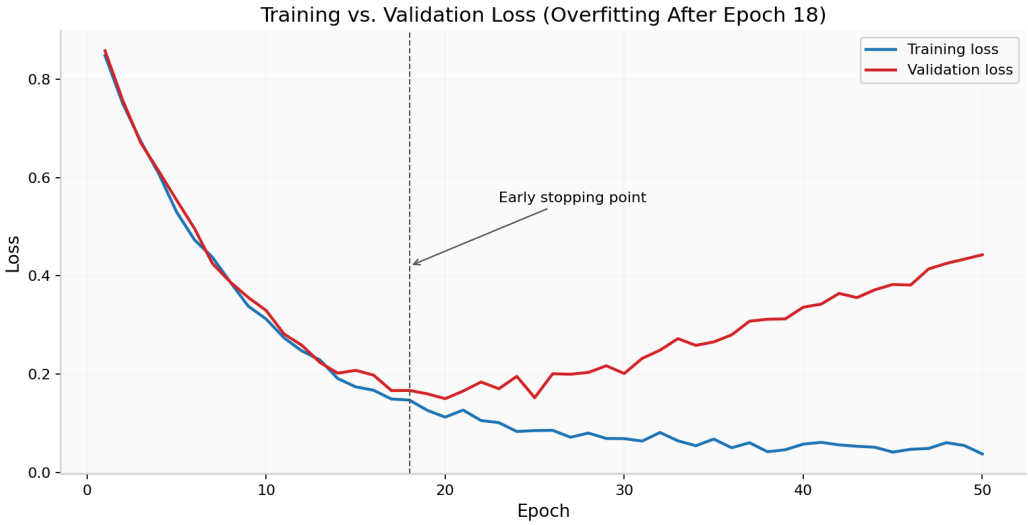


Figure 9. Training and validation loss. Training loss keeps falling while validation loss rises after epoch ~18 — the signature of overfitting, and the signal to stop (checkpoint at the minimum).

14.4 When *not* to use deep learning

Deep learning shines on unstructured data (images, text, audio) at scale. On small tabular datasets it usually *loses* to gradient boosting while costing more to tune and operate. Choose the simplest model that meets the target — that is an engineering virtue, not a lack of ambition.

15 Advanced Architectures: CNNs, RNNs, Transformers

15.1 Architecture is inductive bias

Each architecture bakes in an assumption ("inductive bias") about the data that makes learning dramatically more sample-efficient:

Table 10. Deep learning architectures

Architecture	Inductive bias	Mechanism	Domains
CNN (convolutional network)	Locality + translation invariance	Small filters slide across the input, detecting the same pattern anywhere	Images, video, signals
RNN / LSTM	Sequential order	Hidden state carried step to step; LSTM gates fight vanishing gradients	Time series, classic NLP
Transformer	Learned pairwise relations	Self-attention: every position attends to every other; fully parallel	Language, vision, code, protein folding

15.2 Attention, the idea that ate AI

Self-attention lets each token in a sequence compute a weighted combination of all other tokens, where the weights are learned similarity scores: "which other words matter for understanding *this* one?" Formally, with queries Q, keys K, values V:

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Stacked into multi-head layers with feed-forward blocks and residual connections, this is the Transformer,(6) introduced by Vaswani et al. in 2017[10] and the foundation of modern large language models. Because attention is parallel (unlike RNNs, which must process tokens one at a time), Transformers scale to enormous datasets and parameter counts — and scaling, it turned out, unlocked emergent capabilities.

15.3 Transfer learning and fine-tuning

The most important practical technique in modern ML: **do not train from scratch**. Take a model pretrained by someone else on massive data (a vision backbone, a language model) and *fine-tune* it on your much smaller dataset:

- **Feature extraction:** freeze the backbone, train only a small new head — works with hundreds of examples.
- **Full fine-tuning:** update all weights with a small learning rate — needs thousands of examples.
- **Parameter-efficient fine-tuning (LoRA):** train tiny low-rank adapter matrices instead of the full weights — the standard way to adapt LLMs cheaply.

Example 5 — A text classifier in five lines with a pretrained model

```
from transformers import pipeline

clf = pipeline("text-classification",
               model="distilbert-base-uncased-finetuned-sst-2-english")
print(clf("The deployment went flawlessly and latency dropped 40%."))
# [{'label': 'POSITIVE', 'score': 0.9998}]
```

A model with tens of millions of parameters, pretrained on a large corpus, applied in five lines — the leverage that transfer learning gives a small team would have been unthinkable a decade ago.

15.4 Generative AI in one paragraph

The same Transformer, trained to predict the next token on much of the public internet, becomes a **large language model** capable of reasoning, translation, and code. Combined with instruction tuning and RLHF, such models power copilots, agents, and retrieval-augmented generation (RAG). For operations this creates a new workload class — GPU clusters, vector databases, token-based costs — exactly where Part III picks up.

Part III — Where the Two Meet: MLOps

A model in a notebook creates no value. MLOps applies everything from Part I — automation, containers, CI/CD, observability — to the unique lifecycle of machine learning systems.

16 MLOps: Operating Models in Production

16.1 Why ML systems are harder to operate

A traditional service fails loudly: it crashes, errors spike, someone gets paged. An ML system can fail *silently*: the service returns HTTP 200 with confident, wrong predictions. Google researchers famously described ML systems as carrying enormous "hidden technical debt" because the model is only a small fraction of the real system.[11] The distinctive failure modes:

- **Data drift:** the world changes — input distributions shift (user behavior after a redesign, economic conditions after a rate hike) and model quality decays without any code changing.
- **Training/serving skew:** features computed one way in training and slightly differently in production; among the most common causes of "great offline metrics, terrible live metrics."
- **Feedback loops:** a recommender shapes what users see, which shapes future training data.
- **Stale pipelines:** the model is fine but the feature pipeline silently broke three days ago.

16.2 The MLOps stack

Table 11. MLOps components and representative tools

Component	Purpose	Representative tools
Feature store	One definition of each feature, reused for training and serving (kills skew)	Feast, Tecton
Experiment tracking	Record every run: code, data version, hyperparameters, metrics	MLflow, Weights & Biases
Model registry	Versioned model artifacts with stage labels (staging → production)	MLflow Registry, SageMaker Registry
Training pipeline	Reproducible, schedulable end-to-end training (data → model → evaluation)	Kubeflow Pipelines, Airflow, Metaflow
Model serving	Low-latency prediction endpoints, autoscaling, canary releases	KServe, Seldon, TorchServe, vLLM

Monitoring	Prediction distributions, drift statistics, ground-truth quality when labels arrive	Evidently, WhyLabs, Prometheus + custom metrics

16.3 Serving a model as a containerized API

The simplest robust deployment pattern wraps the model in a small web service — exactly the container and Kubernetes machinery from Part I:

serve.py — model behind FastAPI

```
import joblib
from fastapi import FastAPI
from pydantic import BaseModel

app   = FastAPI()
model = joblib.load("model_v3.joblib")   # artifact from the registry

class Features(BaseModel):
    sepal_length: float
    sepal_width:  float
    petal_length: float
    petal_width:  float

@app.get("/health")
def health(): return {"status": "ok", "model": "v3"}

@app.post("/predict")
def predict(f: Features):
    X = [[f.sepal_length, f.sepal_width, f.petal_length, f.petal_width]]
    cls = int(model.predict(X)[0])
    proba = float(model.predict_proba(X).max())
    return {"class": cls, "confidence": round(proba, 4)}
```

Containerize it with the Dockerfile from Chapter 4, deploy with the Deployment from Chapter 4, and ship updates through the CI/CD pipeline from Chapter 5 — with one ML-specific addition: the pipeline must also run **model validation** (compare candidate vs. champion on a golden dataset) before promotion, and rollbacks are instant because the registry keeps every prior version.

16.4 Monitoring for drift

Log every prediction's input summary statistics. Compare the live feature distribution against the training distribution with a divergence metric (population stability index, KL divergence) on a schedule; alert when it crosses a threshold; trigger automatic retraining when ground-truth labels confirm degradation. This closes the loop drawn in Figure 5 — monitoring feeding back into data preparation.

17 Capstone: An End-to-End Project

We assemble every chapter into one realistic project: **a demand-forecasting service for an online shop**, predicting next-day orders per product to drive inventory decisions.

17.1 Phase 1 — Data and baseline (Chapters 9–13)

1. Extract two years of order history from the production database into object storage (a "bronze" raw layer), then clean into a curated table (the "silver/gold" medallion pattern).
2. Engineer features: day-of-week, holidays, promotions, lagged sales (7/28-day moving averages), price changes.
3. Split *by time*, never randomly — train on 2023–2024, validate on early 2025, test on late 2025. Random splits leak the future into training for any time-series problem.
4. Baseline first: "tomorrow = same day last week." Any model must beat this trivial forecast or be rejected.
5. Train gradient boosting (LightGBM) with time-series cross-validation; evaluate with MAPE and, more importantly, the *business* metric: inventory holding cost + stockout cost.

17.2 Phase 2 — Productionize (Chapters 1–8, 16)

1. **Training pipeline:** a scheduled job (Airflow/Kubeflow) retrains weekly on fresh data and pushes a versioned artifact to the model registry only if it beats the current champion on validation.
2. **Infrastructure:** everything in Terraform — VPC, Kubernetes cluster, object storage buckets with lifecycle policies, IAM roles with least privilege.
3. **Serving:** the FastAPI pattern of Section 16.3, containerized, three replicas across two AZs behind a load balancer, autoscaling on request latency.
4. **CI/CD + GitOps:** pull requests run unit tests and a *model validation gate*; merges update the Git repo that Argo CD reconciles into the cluster; canary release at 5% traffic with automatic rollback on error-rate or prediction-quality regression.
5. **Observability:** RED metrics on the API, plus ML-specific dashboards — prediction distribution vs. training distribution, daily drift score, and realized forecast accuracy as actual orders arrive.
6. **SLO:** 99.9% availability and p95 latency under 200 ms; error budget governs release pace.

17.3 Phase 3 — Operate and improve

1. Weekly drift report; automatic retraining trigger when accuracy decays beyond tolerance.
2. Quarterly chaos drill: kill an AZ, verify the service holds (multi-AZ, Chapter 1).
3. Monthly cost review: right-size the cluster, move training jobs to spot instances, archive cold data.
4. Blameless postmortems for every incident, including silent quality regressions.

Notice the shape of the solution: the *modeling* is a few dozen lines of scikit-learn; the remaining hundreds of lines are pipelines, infrastructure, tests, and monitoring. That ratio is the real content of this guide — and the reason cloud operations and machine learning are best learned together.

17.4 Suggested learning path

Table 12. From zero to advanced: a staged path

Stage	Cloud track	ML track	Milestone project
Beginner	Regions/AZs, VMs, object storage, VPC basics	Features/labels, train-test split, linear & logistic regression	Static site on object storage; iris classifier in a notebook
Intermediate	Terraform, Docker, CI pipeline, monitoring basics	Ensembles, cross-validation, metrics beyond accuracy	Containerized API with CI/CD; tuned GBM on a Kaggle dataset
Advanced	Kubernetes, GitOps, SLOs, FinOps, multi-region DR	Deep learning, transfer learning, MLOps stack	The capstone of Section 17, with drift-triggered retraining

References

1. Mell, P., & Grance, T. (2011). *The NIST Definition of Cloud Computing* (NIST Special Publication 800-145). National Institute of Standards and Technology.
2. HashiCorp. (2024). *Terraform Documentation*. https://developer.hashicorp.com/terraform/docs
3. Burns, B., Beda, J., & Hightower, K. (2019). *Kubernetes: Up and Running* (2nd ed.). O'Reilly Media.
4. Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.
5. Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (Eds.). (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media.
6. Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill.
7. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
8. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794).
9. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
10. Vaswani, A., et al. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, 30.
11. Sculley, D., et al. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems*, 28.